

A Wearable Fall Detection System Using Inertial Sensing on ESP32 with Edge Inference on a Raspberry Pi 4

1st Nihar Ranjan Sarker

0112320140

*Dept. of Computer Science and Engineering
United International University
Dhaka, Bangladesh*

2nd Sabbir Sikder

0112310483

*Dept. of Computer Science and Engineering
United International University
Dhaka, Bangladesh*

3rd Rubayet Islam

0112320046

*Dept. of Computer Science and Engineering
United International University
Dhaka, Bangladesh*

4th Lamia Kamal Nusny

0112320145

*Dept. of Computer Science and Engineering
United International University
Dhaka, Bangladesh*

5th Fariha Mubassira Rifa

0112330291

*Dept. of Computer Science and Engineering
United International University
Dhaka, Bangladesh*

Abstract—This paper presents the design, training pipeline, and prototype evaluation of a wearable fall detection system in which an ESP32 microcontroller acts as a sensor node, streaming inertial data over a WiFi/TCP link to a Raspberry Pi 4 that performs classification with an offline-trained Random Forest model. Falls among elderly or at-risk individuals require rapid, autonomous detection and alerting, since delayed assistance significantly worsens outcomes. The ESP32 samples tri-axial acceleration and angular velocity at a fixed 20 Hz rate, applies a five-sample moving-average filter, and transmits the smoothed feature vector to the Pi4, which normalizes it using a previously fit StandardScaler and classifies it with a 50-tree Random Forest. A consecutive-vote confirmation mechanism (five consecutive fall predictions) suppresses transient false triggers before an alert is raised through an onboard buzzer, a vibration motor, and a GSM module that transmits an SMS notification to a designated contact, with a manual cancel button allowing the wearer to abort a false alarm. The classifier was trained and evaluated offline on a labeled inertial dataset of 124,997 samples (117,742 normal-activity samples and 7,255 fall samples) using an 80/20 stratified train-test split. Among five candidate models, a hyperparameter-tuned Random Forest achieved the best test-set performance, reaching 97.26% overall accuracy, 93.43% fall-detection precision, and 99.75% specificity, while recording 56.86% sensitivity (recall) on the held-out fall class. A breadboard prototype of the full pipeline successfully detected controlled drop tests, but revealed that the WiFi/TCP link between the ESP32 and the Pi4 was unreliable under physical movement, with intermittent data loss traced to loose breadboard wiring rather

than to the classifier itself. These findings, together with a feature-importance analysis identifying vertical and lateral acceleration as the dominant predictive signals, are discussed in the context of the class imbalance inherent to fall datasets and the connection-reliability challenges of a wearable device that must remain functional while undergoing the very motion it is designed to detect.

Index Terms—Fall Detection, Wearable Sensing, ESP32, MPU6050, Inertial Measurement Unit, Random Forest, Edge Computing, Raspberry Pi, Wireless Sensor Networks, GSM Alerting, Assistive Technology

I. INTRODUCTION

[Problem Statement] Falls are among the leading causes of injury and loss of independence for elderly individuals and patients recovering from mobility-impairing conditions. A fall that goes unnoticed for an extended period can result in prolonged immobilization, secondary complications, or, in severe cases, death. Conventional fall-response solutions rely on a bystander noticing the incident or on the affected individual manually triggering an alert, both of which fail precisely in the scenario that matters most: when the person who has fallen is unconscious, disoriented, or otherwise unable to call for help. A dependable fall detection system must therefore operate autonomously, locally, and continuously, without requiring the wearer to take any action at the moment of the fall.

[Literature Review] Prior wearable fall-detection approaches generally fall into two categories: threshold-based methods that flag a fall when raw acceleration magnitude exceeds a fixed empirical cutoff, and machine-learning-based methods that learn a decision boundary from labeled inertial data. Threshold-based methods are computationally trivial but are prone to false positives during vigorous activities (jumping, sitting down quickly) and false negatives during slow, low-impact falls. Machine-learning approaches trained on accelerometer and gyroscope features generally improve discrimination between fall and non-fall motion patterns, but most reported systems perform inference on a paired smartphone or a remote cloud backend, introducing a communication dependency and added latency between the physical event and the alert. This motivates an architecture in which the wearable’s own local network link, rather than the public internet, carries sensor data to a nearby edge-computing device for low-latency inference.

[Innovative Features] The system presented in this paper splits responsibility between a lightweight wearable sensor node and a local edge-computing inference server. An ESP32 microcontroller, fitted with an MPU6050 IMU, samples acceleration and angular velocity at a fixed interval and streams the resulting feature vector over a WiFi/TCP socket to a Raspberry Pi 4 on the same local network, which classifies each sample using a 50-tree Random Forest trained offline in Python. Because the Random Forest was additionally exported into a header-only C++ representation of each tree’s split structure, the same model can be executed directly on the ESP32 without any external machine-learning runtime, making fully on-device inference a supported deployment option even though the prototype evaluated in this paper used the Pi4 as the inference server. A consecutive-vote confirmation window reduces the impact of single-sample misclassifications, and the alert pathway combines an audible buzzer, a vibration motor, and an independent GSM module for SMS notification, with a physical cancel button giving the wearer the ability to suppress a false alarm before a message is sent.

II. PROPOSED METHOD

III. IMPLEMENTED HARDWARE AND SOFTWARE SYSTEM

The system separates concerns between continuous sensor acquisition, embedded statistical inference, and multi-channel alerting, all coordinated by a single microcontroller acting as both the sensing hub and the decision engine.

A. Component Names and Technical Uses

- **ESP32 DevKit:** Wearable sensor node. Samples the IMU over I2C, applies the moving-average filter, and streams the smoothed six-element feature vector to the Raspberry Pi 4 over a WiFi/TCP socket. Also drives the buzzer, vibration motor, and GSM module once a fall is confirmed.
- **MPU6050 Six-Axis IMU:** Provides tri-axial accelerometer readings (configured to a $\pm 8g$ range) and tri-axial gyroscope readings (configured to a $\pm 500^\circ/\text{s}$ range),

digitally filtered on-chip with a 21 Hz bandwidth low-pass filter before being read over the I2C bus.

- **Raspberry Pi 4 (Edge Inference Server):** Receives the streamed feature vector from the ESP32 over the local WiFi network, normalizes it using the StandardScaler parameters fit during offline training, classifies it with the trained Random Forest, and returns a fall/normal prediction to the ESP32.
- **Buzzer and Vibration Motor:** Provide the immediate local alert, giving the wearer audible and tactile feedback that a fall has been detected and that an alert is pending.
- **Cancel Button:** A debounced push-button input that allows the wearer to abort a triggered alert before the SMS notification is sent, or to silence the alert after the fact.
- **GSM Module:** Connected over a software serial bridge, used to transmit an SMS notification to a pre-configured emergency contact number when a fall is confirmed and not cancelled within the alert window.
- **Random Forest Classifier:** A 50-tree ensemble model, trained offline on labeled IMU data. In the evaluated prototype this model runs on the Raspberry Pi 4 using scikit-learn; the same model was additionally exported as a header-only C++ representation (split feature index, threshold, and child-node arrays per tree), making standalone on-device inference on the ESP32 a supported but not yet field-tested deployment mode.

B. Hardware Architecture and Electrical Interconnects

- **ESP32 $\leftrightarrow$ MPU6050 (I2C Bus):** SDA $\rightarrow$ GPIO 21, SCL $\rightarrow$ GPIO 22, VCC $\rightarrow$ 3.3 V, GND $\rightarrow$ GND.
- **ESP32 $\leftrightarrow$ Raspberry Pi 4 (WiFi/TCP Socket):** The ESP32 connects to the same local WiFi network as the Pi4 and opens a TCP client socket to a Python server script running on the Pi4, sending one feature-vector packet per sampling interval and receiving a single-byte fall/normal prediction in response.
- **Buzzer:** Signal $\rightarrow$ GPIO 4.
- **Vibration Motor:** Signal $\rightarrow$ GPIO 5.
- **Cancel Button:** Signal $\rightarrow$ GPIO 15, configured as INPUT_PULLUP so the resting state reads high and a press pulls the line low.
- **Pulse Sensor (provisioned, optional):** Analog output $\rightarrow$ GPIO 34 (ADC1 channel), reserved for future heart-rate-assisted alert confirmation.
- **GSM Module (Software Serial Bridge):** ESP32 GPIO 16 $\rightarrow$ GSM RX, ESP32 GPIO 17 $\rightarrow$ GSM TX, communicating at 9600 bps using AT commands in text SMS mode (AT+CMGF=1).

C. Feature Vector and Sampling Pipeline

At every control loop iteration the ESP32 firmware reads a six-element feature vector directly from the MPU6050:

$$\mathbf{x} = [a_x, a_y, a_z, \omega_x, \omega_y, \omega_z] \quad (1)$$

where a_x, a_y, a_z are the acceleration components expressed in units of standard gravity $g = 9.81 \text{ m/s}^2$, and $\omega_x, \omega_y, \omega_z$ are the angular velocity components in degrees per second. Each component is passed through a five-sample circular-buffer moving average on the ESP32 before being transmitted:

$$\bar{x}_i = \frac{1}{5} \sum_{k=0}^4 x_i[n-k] \quad (2)$$

to suppress high-frequency sensor noise prior to transmission and classification.

D. Feature Normalization

Because the Random Forest was trained on standardized features, the smoothed feature vector received from the ESP32 is normalized on the Raspberry Pi 4 using the mean and standard deviation learned by a StandardScaler during offline training:

$$z_i = \frac{\bar{x}_i - \mu_i}{\sigma_i}, \quad i = 1, \dots, 6 \quad (3)$$

The same scaler parameters were also exported as constant C++ arrays for the standalone on-device deployment mode described in Section III-F.

E. Sensor Acquisition and Smoothing

The firmware reads the MPU6050 directly through the Adafruit sensor library and converts the raw accelerometer output into units of standard gravity before the five-sample moving-average filter is applied, exactly as implemented in the deployed sketch:

```

1 void readSensors(float features[]) {
2   sensors_event_t a, g, temp;
3   mpu.getEvent(&a, &g, &temp);
4
5   features[0] = a.acceleration.x / GRAVITY;
6   features[1] = a.acceleration.y / GRAVITY;
7   features[2] = a.acceleration.z / GRAVITY;
8   features[3] = g.gyro.x;
9   features[4] = g.gyro.y;
10  features[5] = g.gyro.z;
11 }
12
13 void applyMovingAverage(float features[]) {
14   accelXBuffer[bufferIndex] = features[0];
15   accelYBuffer[bufferIndex] = features[1];
16   accelZBuffer[bufferIndex] = features[2];
17   gyroXBuffer[bufferIndex] = features[3];
18   gyroYBuffer[bufferIndex] = features[4];
19   gyroZBuffer[bufferIndex] = features[5];
20
21   float sumX=0, sumY=0, sumZ=0, sumGX=0, sumGY=0,
22     sumGZ=0;
23   for (int i = 0; i < 5; i++) {
24     sumX += accelXBuffer[i]; sumY += accelYBuffer[i];
25     sumZ += accelZBuffer[i]; sumGX += gyroXBuffer[i];
26     sumGY += gyroYBuffer[i]; sumGZ += gyroZBuffer[i];
27   }
28   features[0] = sumX / 5.0; features[1] = sumY /
29     5.0;

```

```

29   features[2] = sumZ / 5.0; features[3] = sumGX /
30     5.0;
31   features[4] = sumGY / 5.0; features[5] = sumGZ /
32     5.0;
33   bufferIndex = (bufferIndex + 1) % 5;
34 }

```

Listing 1. Sensor acquisition and moving-average smoothing, excerpted from the deployed ESP32 firmware (ESP32_Fall_Detection.ino).

After smoothing, the resulting six-value feature vector is serialized and transmitted over the WiFi/TCP socket to the Raspberry Pi 4, which performs normalization and classification as described in Section III-E and returns the prediction.

F. Random Forest Inference: Edge Server and On-Device Export

In the evaluated prototype, classification is performed on the Raspberry Pi 4 using the trained scikit-learn RandomForestClassifier directly: the Pi4's TCP server receives the smoothed feature vector from the ESP32, applies the StandardScaler transform, calls `model.predict()`, and returns the resulting label over the same socket.

The same trained model was separately exported into a header-only C++ class so that the identical decision logic can, in principle, run directly on the ESP32 without any external machine-learning library. Each tree is represented as four parallel arrays (left-child index, right-child index, split feature index, split threshold) plus a leaf-value table, and feature normalization and classification follow the same two-step pattern as the Pi4 path:

```

1 void normalizeFeatures(float raw[], float normalized
2   []) {
3   for (int i = 0; i < ScalerParams::NUM_FEATURES; i
4     ++){
5     normalized[i] = (raw[i] - ScalerParams::MEAN[i])
6       / ScalerParams::STD[i];
7   }
8 }
9
10 int detectFall(float features[]) {
11   float normalized[ScalerParams::NUM_FEATURES];
12   normalizeFeatures(features, normalized);
13   return FallDetectionModel::predict(normalized);
14 }

```

Listing 2. Feature normalization and fall classification wrapper for the standalone on-device deployment mode, excerpted from the exported firmware.

The `FallDetectionModel::predict()` call walks all 50 trees of the exported Random Forest and returns a majority vote across the ensemble, requiring no dynamic memory allocation and no external inference runtime. This on-device path was compiled and verified to fit within the ESP32's flash and RAM budget, but the system evaluated end-to-end in this paper used the Pi4 edge-server path; standalone on-device inference, and a direct comparison of its latency and reliability against the WiFi/TCP path, remain future work.

G. Consecutive-Vote Confirmation and Alert State Machine

A single misclassified sample is insufficient to trigger an alert. Whichever inference path is active (Pi4 edge server

or standalone on-device), the ESP32 accumulates consecutive “fall” predictions in a running counter and only confirms a fall once `FALL_DETECTION_THRESHOLD` (five) consecutive positive predictions have occurred, resetting the counter once enough consecutive “normal” predictions are observed. This logic is implemented in the main control loop of the standalone on-device sketch, which runs at a fixed 50 ms interval (20 Hz) and is structurally identical to the polling loop used in the WiFi/TCP variant, with the local `detectFall()` call replaced by a socket round-trip to the Pi4:

```

1 void loop() {
2   float features[6];
3   bool cancelled = cancelAlert();
4
5   readSensors(features);
6   applyMovingAverage(features);
7
8   int prediction = detectFall(features);
9   if (prediction == 1) {
10    fallVoteCount++;
11    normalVoteCount = 0;
12  } else {
13    normalVoteCount++;
14    if (normalVoteCount > 3) {
15      fallVoteCount = 0;
16    }
17  }
18
19  if (!fallDetected && fallVoteCount >=
20      FALL_DETECTION_THRESHOLD) {
21    fallDetected = true;
22    fallDetectedTime = millis();
23    triggerFallAlert();
24  }
25
26  if (fallDetected && !smsSent && !cancelled) {
27    if (millis() - fallDetectedTime >= SMS_DELAY_MS)
28    {
29      sendSMS();
30      smsSent = true;
31    }
32  }
33  delay(100);
34 }

```

Listing 3. Main control loop showing the consecutive-vote logic and alert trigger, from the standalone on-device firmware variant.

In the WiFi/TCP variant used for the breadboard prototype, `detectFall()` is replaced by a function that opens (or reuses) a TCP connection to the Pi4, writes the smoothed feature vector, and blocks briefly to read back a single-byte prediction; the vote-counting and alert logic shown above is otherwise unchanged.

H. Breadboard Prototype Testing

The complete pipeline, ESP32 sensor node, WiFi/TCP link, and Pi4 inference server, was assembled on a breadboard and tested with controlled drop tests in which the device was dropped from a stationary hand-held position to simulate a fall impulse. Under these conditions the system correctly registered a fall and progressed through the consecutive-vote and alert sequence. However, this breadboard-stage testing also surfaced a hardware reliability issue distinct from the classifier’s own performance: when the assembly was subjected

to sustained physical movement, rather than a single drop, the WiFi/TCP link between the ESP32 and the Pi4 became unreliable, intermittently failing to transmit feature vectors or receive predictions. This behavior is discussed further in Section VI.

Once a fall is confirmed, the system:

- 1) Immediately drives the buzzer and vibration motor in a fast-pulse alert pattern.
- 2) Waits a configurable grace period (`SMS_DELAY_MS`, 5 seconds) during which the cancel button can abort the alert.
- 3) If not cancelled, sends an SMS notification through the GSM module to a pre-configured contact number.
- 4) Continues alerting until the cancel button is pressed, which resets the vote counters and returns the system to normal monitoring.

IV. DATASET AND OFFLINE MODEL TRAINING

A. Dataset Composition

The classifier was trained on a labeled inertial dataset combining controlled fall trials (forward, left, and right falls from both standing and sitting postures) with a broad range of non-fall activities (standing, sitting, walking, and additional incidental motion recordings). After removing rows with missing sensor values and discarding statistical outliers (z -score > 4 on any sensor channel), the cleaned dataset contained 124,997 labeled samples across six numeric features (`AccX`, `AccY`, `AccZ`, `GyroX`, `GyroY`, `GyroZ`), with a binary label assigned by string-matching each row’s original activity tag for the substring “fall.” The resulting class distribution is summarized in Table I.

TABLE I
CLASS DISTRIBUTION AFTER CLEANING

Class	Samples	Proportion
Normal Activity (0)	117,742	94.19%
Fall (1)	7,255	5.81%
Total	124,997	100%

This imbalance, characteristic of fall-detection datasets in which fall events are necessarily rare relative to ordinary daily activity, is an important factor in interpreting the model’s downstream sensitivity, discussed in Section VI.

B. Model Selection

Five candidate classifiers were trained on an 80/20 stratified train-test split (99,997 training samples, 25,000 test samples) using the six standardized features as input: a single Decision Tree, a Random Forest (100 estimators, default depth), a Gradient Boosting ensemble, a Support Vector Machine with an RBF kernel, and a Multi-Layer Perceptron with two hidden layers (64 and 32 units). Table II reports test-set accuracy and F1-score for each candidate prior to hyperparameter tuning.

TABLE II
MODEL COMPARISON ON THE HELD-OUT TEST SET

Model	Accuracy	F1-Score
Random Forest	97.28%	0.7116
SVM (RBF)	97.19%	0.6932
Neural Network (MLP)	97.18%	0.7121
Gradient Boosting	97.12%	0.6926
Decision Tree	95.35%	0.6093

Random Forest was selected for deployment based on its leading accuracy and F1-score, combined with its tree-structured decision logic being straightforward to export as static arrays for direct execution on the ESP32, unlike the SVM and MLP candidates, which would require floating-point kernel or matrix-multiplication routines on-device.

C. Hyperparameter Tuning

A grid search over 108 parameter combinations ($n_estimators \in \{50, 100, 200\}$, $max_depth \in \{10, 20, 30, None\}$, $min_samples_split \in \{2, 5, 10\}$, $min_samples_leaf \in \{1, 2, 4\}$) was performed with five-fold cross-validation on the training set, optimizing for accuracy. The search identified $n_estimators = 50$, $max_depth = 20$, $min_samples_split = 10$, and $min_samples_leaf = 1$ as the best-performing configuration, with a cross-validated training accuracy of 97.27%. Notably, the tuned model uses only 50 trees rather than the 100 used in the initial comparison, which directly benefits the firmware’s memory footprint on the ESP32 without materially affecting accuracy.

V. RESULTS

A. Test-Set Performance

The tuned Random Forest was evaluated on the untouched 25,000-sample test set, achieving 97.26% overall accuracy and an F1-score of 0.7069 on the fall class. The corresponding confusion matrix is shown in Table III.

TABLE III
CONFUSION MATRIX ON THE TEST SET ($n = 25,000$)

		Predicted	
		Normal	Fall
Actual	Normal	23,491	58
	Fall	626	825

From this confusion matrix, three operationally relevant metrics were derived:

- **Sensitivity (fall recall):** $\frac{825}{825 + 626} = 56.86\%$
- **Specificity (normal recall):** $\frac{23,491}{23,491 + 58} = 99.75\%$
- **Precision (fall class):** $\frac{825}{825 + 58} = 93.43\%$

B. Feature Importance

Fig. 1 reports the Random Forest’s impurity-based feature importance, computed over the six input features. Vertical acceleration ($AccY$) is the single most discriminative feature (importance 0.314), followed by $AccX$ (0.201), $GyroZ$ (0.174), and $AccZ$ (0.160), with the remaining gyroscope axes contributing comparatively little ($GyroY$: 0.078, $GyroX$: 0.073). This ranking is consistent with the physical intuition that a fall is dominated by a rapid, large-magnitude change in linear acceleration along the body’s primary vertical and forward axes, with rotational velocity playing a secondary, confirmatory role.

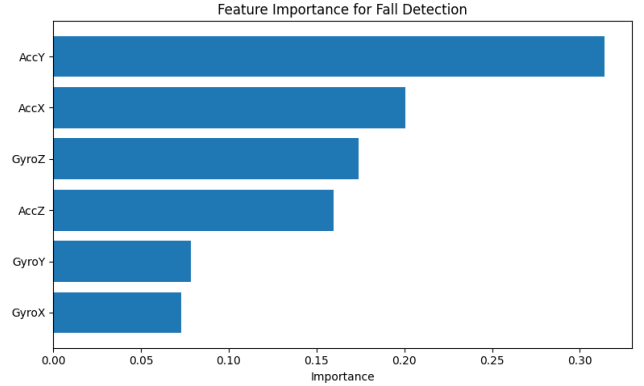


Fig. 1. Random Forest feature importance across the six IMU-derived input features.

VI. DISCUSSION

An evaluation of the system’s constraints and engineering trade-offs reveals several key factors relevant to deployment as a wearable safety device.

Class Imbalance and the Sensitivity–Precision Trade-off: The dataset’s natural imbalance (94.19% normal activity versus 5.81% fall samples) means that overall accuracy is a misleading headline metric: a trivial classifier that always predicts “normal” would already score above 94%. The deployed model’s 56.86% sensitivity indicates that, on this held-out test set, just over half of true fall events are correctly flagged in a single classification pass, while its 99.75% specificity and 93.43% precision indicate that false alarms on normal activity are rare. The firmware’s five-consecutive-vote confirmation logic is a deliberate mitigation for this asymmetry: because a genuine fall produces a sustained burst of abnormal readings across multiple consecutive 50 ms sampling cycles rather than a single instantaneous spike, the effective sensitivity of the complete system (sensor plus voting logic) is expected to exceed the single-sample sensitivity reported here, though this has not yet been independently validated against a labeled stream of full fall sequences on hardware.

Limitations and Design Constraints: The MPU6050 alone cannot distinguish a genuine fall from other abrupt motions that share a similar acceleration signature, such as dropping the device, sitting down rapidly, or running. The reliance on

only six raw kinematic features, without barometric altitude or orientation-fusion data, was a deliberate simplification to keep the on-device model small enough to execute as static C++ arrays within the ESP32's available flash and RAM, at some cost to discriminative power relative to richer feature sets reported elsewhere in the literature.

Wireless Link Reliability Under Physical Motion: It is important to separate the performance of the Random Forest classifier itself, which performed well both offline and in controlled drop tests, from the reliability of the communication link carrying sensor data to it. The breadboard prototype's WiFi/TCP connection between the ESP32 and the Raspberry Pi 4 was stable while the assembly was stationary, but became intermittent once the assembly was subjected to sustained physical movement: feature-vector packets were occasionally not transmitted or not received, and the corresponding prediction round-trip was lost. This is consistent with loose breadboard jumper-wire connections being disturbed by motion, the very condition under which the device most needs to keep working, rather than with any deficiency in the WiFi protocol, the TCP socket logic, or the classifier. The finding nonetheless highlights a genuine engineering requirement for a wearable safety device of this kind: the physical connections between the sensor node, any board-level wiring, and (if a split architecture is retained) the network link, must be mechanically robust to vibration and impact, which a breadboard prototype by its nature cannot guarantee. A soldered or PCB-mounted ESP32 module, and either a more robust wireless protocol or the standalone on-device inference path described in Section III-F (which removes the wireless dependency entirely), are both direct mitigations suggested by this finding.

Conflicting Requirements and Alternatives: Splitting inference onto the Raspberry Pi 4 permits a model that runs through the full scikit-learn library rather than a hand-exported static representation, and simplifies iterating on the classifier without reflashing the ESP32. The cost of this choice, observed directly in the breadboard testing described above, is a dependency on a continuously available WiFi link between the two devices, which proved to be the least reliable part of the system under motion. The standalone on-device export described in Section III-F was prepared specifically as a mitigation for this dependency, and is the natural next deployment target now that the link-reliability issue has been identified.

Interdependence: The alert pathway is intentionally layered: the buzzer and vibration motor fire immediately and unconditionally upon a confirmed fall, independent of whether the GSM module successfully registers on the cellular network, ensuring that a local audible alert is never silently dependent on telecom connectivity.

VII. CONCLUSION

This paper presented a wearable fall detection system in which an ESP32 sensor node streams smoothed inertial features to a Raspberry Pi 4 edge server for classification by a Random Forest model trained offline on a 124,997-sample labeled inertial dataset. The classifier itself performed well,

achieving 97.26% test accuracy, 93.43% fall-detection precision, and 99.75% specificity, with a single-sample sensitivity of 56.86% that the system's consecutive-vote confirmation logic is designed to compound favorably across a sustained fall event. A breadboard prototype of the full pipeline correctly detected controlled drop tests, but also revealed that the WiFi/TCP link between the ESP32 and the Pi4, rather than the classifier, was the weakest link in the system: physical movement disturbed the breadboard wiring and caused intermittent data loss. This points clearly to the system's most important next step, which is not further model tuning but mechanically robust wiring (a soldered or PCB-mounted assembly) and validation of the standalone on-device inference path already exported for the ESP32, which would remove the wireless dependency identified in this prototype entirely. Beyond this, future iterations will focus on validating end-to-end sensitivity against live, full-length fall sequences captured directly from the deployed hardware, exploring windowed time-series features (such as jerk and signal-magnitude-area over a sliding window) to recover sensitivity without enlarging the on-device model, and extending the alert pathway with GPS-based location reporting alongside the existing SMS notification.

ACKNOWLEDGMENT

The development team extends sincere gratitude to the faculty supervisors and laboratory staff who provided the hardware, testing environment, and technical guidance required to construct and validate this fall detection system.

REFERENCES

- [1] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [2] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [3] M. Mubashir, L. Shao, and L. Seed, "A survey on fall detection: Principles and approaches," *Neurocomputing*, vol. 100, pp. 144–152, 2013.
- [4] Project Repository: <https://github.com/emailidnrs-droid/Fall-Detection-System>